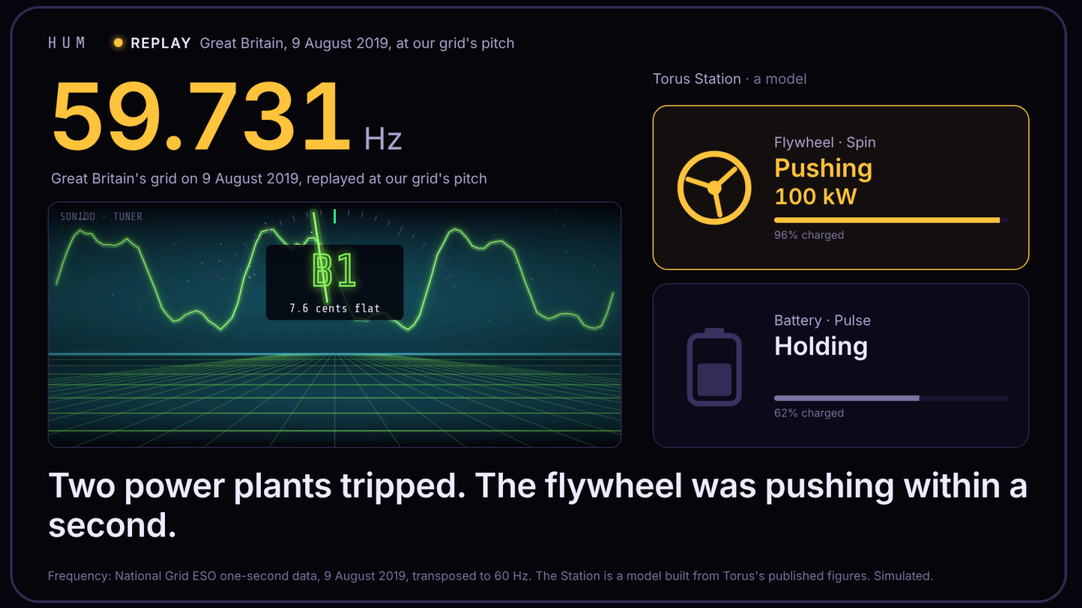
Hard Tech Week 2026 · Torus bounty · Nucleus × Startup State open call

# HUM hears the power grid through a guitar, and a model of a Torus Station answers it.

The software that decides is written in Mentl, a language I built. Its rules are in the code, and the compiler checks them through every function the decision calls, so it can't reach the internet while it decides, and a version that breaks a rule does not build.

**Morgan Espitia**, Salt Lake City. My company is Mentl. Its first market: the people who run batteries and flywheels.




The live screens read the grid from a room's wiring, except the replayed 2019 event.

ampactor.dev/hum

Data: grid frequency, NESO open data at 1 s, 9 Aug 2019, moved to 60 Hz · rush hour, EIA-930 hourly demand, PacifiCorp East, 19–26 Oct 2026 · Station: a simulated model of Torus's published figures

How to read this demo

# The power grid's frequency shows whether it has enough power. A guitar can hear it.

**The grid** The whole Western grid, from Alberta to Baja, runs at one frequency: 60 hertz. When demand outruns supply, the frequency falls.

**The ear** Every power adapter hums at that frequency. A guitar pickup resting on one hears it, and my audio library, sonido, measures it to a thousandth of a hertz.

**The Station** A model of a Torus Station: a flywheel for the first seconds (100 kW for six minutes) and a battery for the hours (300 kW for four hours). Simulated.

**The software** Decides what the Station does. It is written in Mentl, a language I built: its rule, never reach the internet, is in the code, and the compiler checks it through every function the decision calls.

**Next** The grid's hum in a room's wiring. Then tomorrow's rush hour, a real grid failure from 2019, and a blackout.



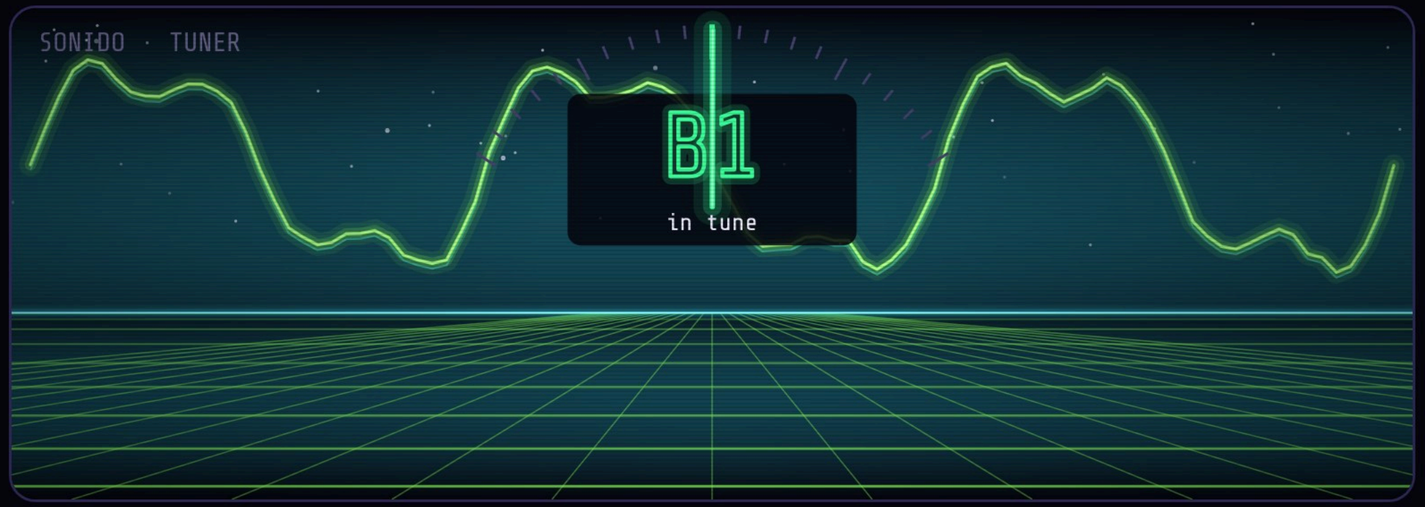
Station figures: Torus's published specifications, modeled and simulated. Frequency: read from a room's wiring.

H U M ○ **TEST HUM** synthesized at 60 Hz

**The grid's note: in this recording, a synthesized stand-in**

# 60.004 Hz

a synthesized hum, standing in for the grid




Torus Station · a model

Flywheel · Spin

**Ready**

94% charged

Battery · Pulse

**Charging**

**130 kW**

62% charged

## The whole Western grid hums one note: 60 hertz.

In the room, a guitar pickup rests on a power adapter, which hums at the grid's frequency, and sonido measures it. This recording uses a synthesized 60 Hz hum in its place. At 60 Hz the tuner reads the note B, in tune. If the grid ran short of power, the number would fall.

The Station: a model from Torus's published figures, simulated.

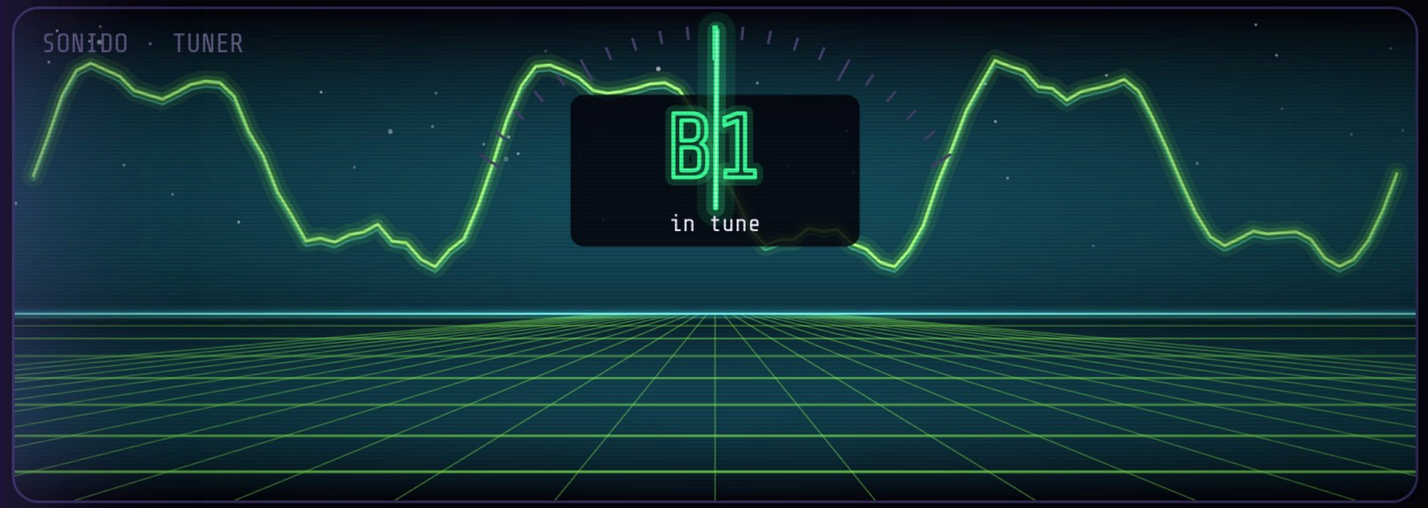
HUM ○ TEST HUM synthesized at 60 Hz

A jam you can see coming: tomorrow's rush hour

TOMORROW · 12:30 PM

# 60.014 Hz

a synthesized hum, standing in for the grid




Torus Station · a model

Flywheel · Spin

**Ready**

95% charged

Battery · Pulse

**Running the building**

**300 kW**

62% charged

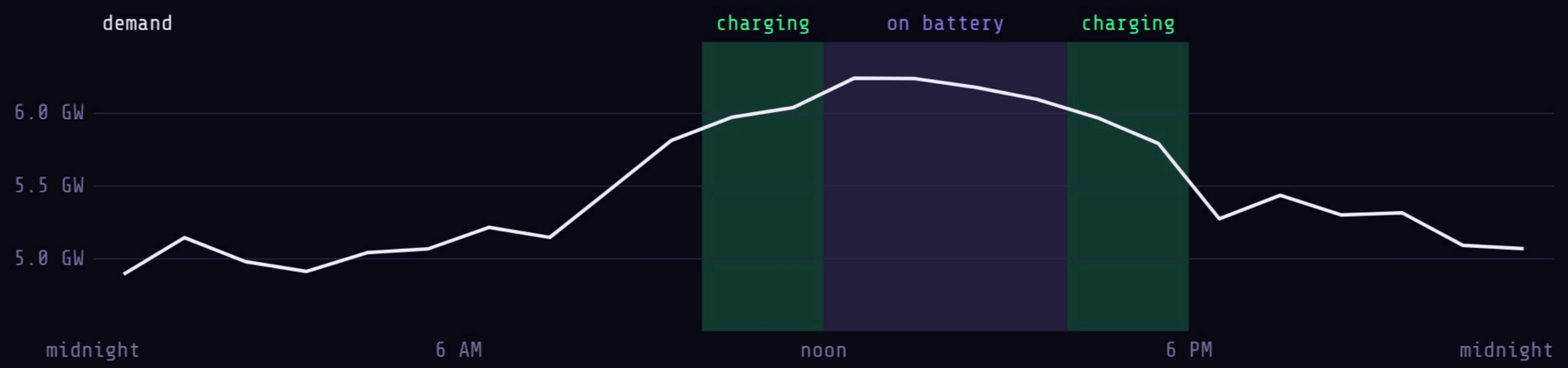
## Rush hour. The Station runs the building on its battery: 300 kW kept off the grid while demand peaks.

EIA's hourly data shows PacifiCorp East's demand peaking from noon to 4 PM this week. At tomorrow's 12:30 PM the model's building draws its rush-hour 300 kW, and the battery carries all of it, so none of it comes from the grid.

Rush hour: EIA-930 hourly demand for PacifiCorp East, 19–26 Oct 2026; tomorrow is the mean of the last seven days. The Station: a model from Torus's published figures, simulated.

## Tomorrow, hour by hour

# Through rush hour the battery carries the building. In cleaner hours it recharges.



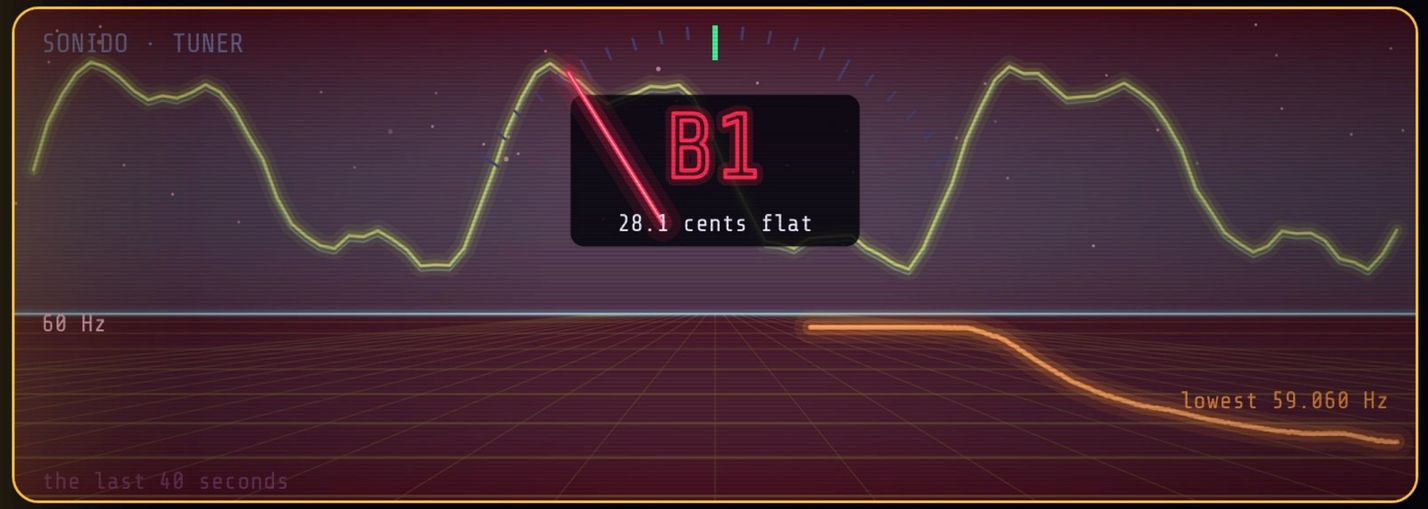

PacifiCorp East is the balancing area that covers most of Utah. EIA's hourly data puts tomorrow's highest demand from noon to 4 PM. In those hours the model's battery carries the building's whole 300 kW. Outside them, in any hour when at least 40 percent of the power is clean, it recharges. These are the controller's own rules.

Demand: EIA-930 hourly data for PacifiCorp East; tomorrow is the mean of the last seven days. Clean share: EIA-930 generation by fuel (nuclear, hydro, solar, wind and geothermal). The Station is a model, simulated.

**A jam you can only hear: a real failure, replayed**

# 59.060 Hz

Great Britain's grid on 9 August 2019, replayed at our grid's pitch




Torus Station · a model

Flywheel · Spin

**Pushing**

**100 kW**

93% charged

Battery · Pulse

**Holding**

62% charged

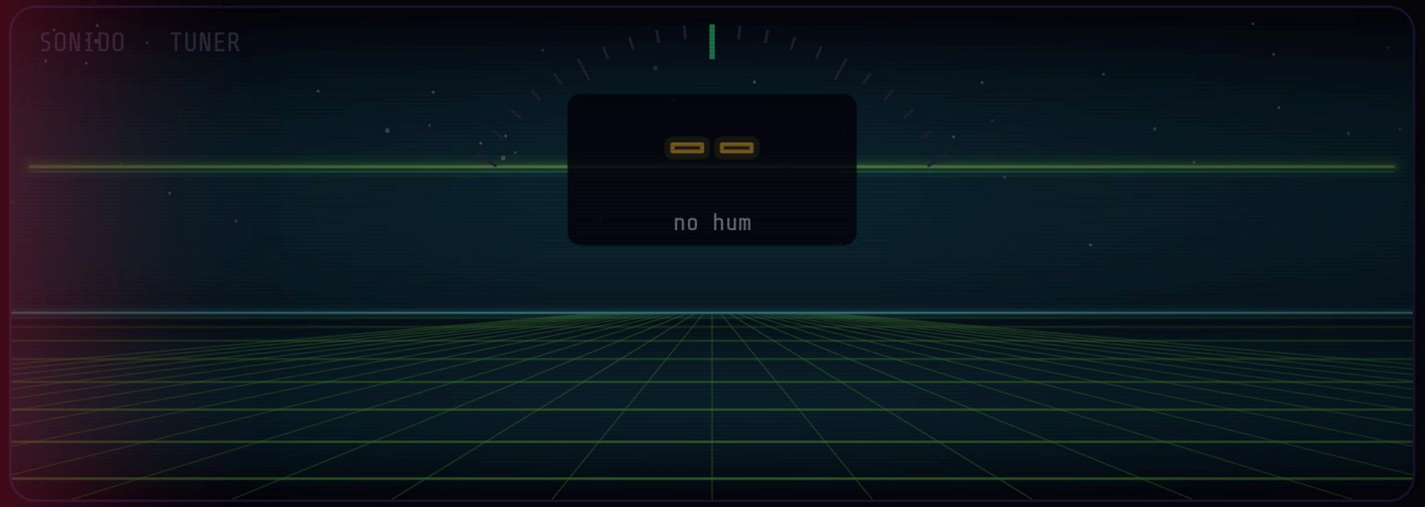
## Two power plants tripped. The flywheel was pushing within a second.

On 9 August 2019, two power plants in Britain dropped off within a second and the frequency fell. This is that day's recorded frequency, replayed at 60 Hz. The flywheel starts pushing about 0.4 seconds after the fall shows and is at full power within 0.75 seconds. In this model the flywheel answers the seconds, and the battery keeps its charge for the hours.

Frequency: NESO open data at one second, Great Britain, 9 August 2019, transposed to 60 Hz. The Station: a model from Torus's published figures, simulated.

# no hum

the hum is cut, as if the grid went dark




Torus Station · a model

Flywheel · Spin

**Ready**

55% charged

Battery · Pulse

**Running the building**

**200 kW**

62% charged

## The grid went dark. The building is still on.

That evening a million people in Britain lost power. Here the hum is cut, as if the grid went dark. The model's flywheel carries the first seconds, then its battery runs the building's 200 kW, its load outside rush hour. A real building also needs a transfer switch; the model assumes one.

The Station: a model from Torus's published figures, simulated.

# ✓ Proven

## This software can't reach the internet while it decides.

Every decision you just watched was made by software written in Mentl, a language I built. Its compiler checks every function the decision calls, and proves the decision can't reach the internet, a file or the clock, and never allocates memory. It acts only on the numbers it is handed. Code added later that tries to reach the internet won't build, as long as the rule stays on the function.

**Not proven:** that its decisions are good. They are tested on recordings.

```
Checked by Mentl's compiler: mentl check examples/grid/station.mn
```

# ✕ Refused

## One line broke the rule.

```
let price = price_m("https://api.eia.gov/…")
```

Say an AI assistant adds this line to the split-second decision, to check electricity prices online. The hour's plan comes from EIA's data, read before the decision runs. The decision itself is marked as never reaching the internet, so the compiler won't build the program. Its error names the path:

```
decide calls price_m → price_m calls fetch_param → fetch_param reaches the Network
```

The compiler's own output: `mentl check examples/grid/station-leak.mn`

# AI now helps write the software that runs machines. C and C++, the languages most of it is written in, can't check what that software will never do.

Batteries, flywheels and flight controls run on code that people test and review, and then trust.

# The rule is part of the code, and the compiler checks it through every call.

You write the rule into the code: this never touches the internet, this never writes a file, this never allocates memory.

The compiler checks the rule through every function the code calls. **Break it, and the program won't build.**

Seven months and 2,600 commits since March. The compiler is written in Mentl and compiles itself. Open source: github.com/ampactor-labs/mentl

The compiler's answer about the controller in this demo

```
$ mentl why examples/grid/station.mn decide Network
decide never uses Network: nothing in decide, and
nothing it calls, performs an operation of Network

$ mentl why examples/grid/station.mn decide Alloc
decide never uses Alloc: nothing in decide, and
nothing it calls, performs an operation of Alloc
```



Counts: the git history of github.com/ampactor-labs/mentl. The answers: pitch/why.txt, captured from the compiler on 28 Oct 2026.

# Everything you just saw was built since Monday at 4 PM, on two tools I built before it.

**Before** Mentl, the language: 2,610 commits since March 15. sonido, the audio library: 461 commits since January 25.

**This week** All of HUM: the ear, the controller, the director, the page and this show. In sonido, 10 commits, among them the grid ear and the pedal. In Mentl, 27.

**Check it** Every commit, with its time: github.com/ampactor-labs/hum, printed by `./ledger.sh`



Counts: each repository's git history since Monday 26 Oct 2026, 4:00 PM MDT, printed in pitch/ledger.txt.

# Who needs it in Utah

**TORUS · SALT LAKE CITY**

Builds flywheel-and-battery Stations that steady the grid. The software inside each Station is where a proof would matter first. Torus is judging this demo.

**HILL AFB · 309TH SOFTWARE ENGINEERING GROUP**

Maintains flight software for the F-16, the F-35 and the missile fleet, where every claim about the software needs evidence.

The sites I'll ask to host the pilot: Torus, Rocky Mountain Power's Wattsmart Battery program, and Utah State University's ASPIRE center.
First hires: a controls engineer and a compiler engineer, in Salt Lake City.

What $10,000 unlocks

# $10,000 puts this controller beside a real battery in Utah within ninety days.

**Day 30** Form the Utah company. Run the controller on a 3 kW inverter and 10 kWh of batteries on my bench.

**Day 60** With a site's written consent, run it read-only beside a real battery for 14 days, logging its decisions and sending no commands.

**Day 90** A pilot agreement that lets it send commands, and an SBIR proposal with Nucleus Grow.

**$7,000** for the bench. **$3,000** for the company, a lawyer for the pilot agreement, a year of insurance and the site's read-only kit. Every line is priced in the budget.

---

**Three asks.** A Utah operator who will host it. An introduction to Hill AFB's 309th. A co-founder who knows power electronics.

Thank you, Torus and Nucleus

# Replay this demo on your phone.



`ampactor.dev/hum`

**Morgan Espitia** · sonofagoose@pm.me

The demo: github.com/ampactor-labs/hum
The language: github.com/ampactor-labs/mentl

Three asks: a Utah operator who will host the pilot, an introduction to Hill AFB's 309th, and a co-founder who knows power electronics.

What Mentl code looks like

# Five operators draw the shape of a program. The compiler checks each rule along that shape.

| | | |
|---|---|---|
| `let l = saw \|> rig.dark(300.0) \|> soft_clip` | `\|>` | in sequence: the sawtooth, the filter, the clipper |
| `let (echoed, roomed) = dry <\| (rig.echo, rig.room)` | `<\|` | one sample to the echo and to the room; shared, so the two cannot race |
| `let l = ((echoed) >< (dry)) \|> mix(0.35)` | `><` | two signals side by side, then mixed |
| `{ prev => a * x + (1.0 - a) * prev } <~ delay(1)` | `<~` | feedback: this sample from the last one; the memory is allocated once |
| `handler rack = spectral_flux_distort ~> limiter_at(0.6)` | `~>` | the handler: what the sound goes through, named once |

The controller's rule is on its decision: `with !Alloc + !Network + !Filesystem + !WASI + !Trap`. The compiler checks it through every function the decision calls, and the copy that read a price online did not build.



Lines from lessons 2, 3 and 4 of the Mentl course and from examples/grid/station.mn, as they check clean on 29 Oct 2026. The language: github.com/ampactor-labs/mentl

BACKUP

# What is proven, exactly

| THE CONTROLLER'S RULE | STATUS |
| --- | --- |
| No network, file, clock or other call outside the program while it decides | Proven by the compiler through every call |
| No memory allocated while it decides | Proven |
| The flywheel within 100 kW, the battery within 300 kW | Proven: each machine answers a command with a power inside its rating, and the compiler checks the answer |
| It never stops with an error | Proven for every function the controller defines; one byte-range check inside the library's print routine is not proven yet |
| Its decisions are good | Tested on recordings, not proven |

The compiler is written in Mentl, and the pinned binary rebuilds itself byte for byte. The first four rows are checked through every call, and a copy that breaks one does not build.

Checked by the compiler: mentl audit examples/grid/station.mn, and mentl why for each rule

BACKUP

# From the laptop to the pedal: the engineering left

| WHAT STANDS IN THE WAY | MEASURED TODAY | THE FIX |
|---|---|---|
| No WebAssembly engine on the pedal's chip | The controller is a 43 KB WebAssembly file | Embed an interpreter written in Rust, or compile the file to C |
| Declared for threads | The memory is declared shared; there is no atomic instruction and no thread import | Drop the word for a program that starts no thread, a one-line change in the compiler |
| Memory | Fixed: the same peak after 300,000 readings as after one | Done; each reading runs in an arena that is reclaimed when it ends |
| Speed | About 291,000 instructions per reading | Pass numbers instead of text |

The decision allocates nothing and the loop reclaims each reading's memory. The work left is the engine on the chip.